

CSE 2216 - Application Development Lab

Project Report

APP LOGO



Project Name: FamNest

Team Name: **TeamExtreme**

Submitted By

Tawyabul Islam Tamim -04

Md. Shahriar Kabir - 20

Ovijit Chandra Balo -28

Faiaz Mahmud Ifti - 46

28th Batch

Department of Computer Science & Engineering
University of Dhaka

Submitted On

2nd February 2025

Table of Contents

1. Introduction	3
1.1 About the App	3
1.2 Motivation	3
1.3 Features (Bullet Points)	3
1.4 Tools, Technologies and Frameworks Used	3
1.5 Individual Responsibilities	3
1.5.1 Tawyabul Islam Tamim - 04	
1.5.2 Md Shahriar Kabir -20	
1.5.3 Ovijit Chandra Balo -28	
1.5.4. Faiaz Mahmud Ifti - 46	3
2. Design and Implementation	4
2.1 UI Developed (Screenshots)	4
2.2 Database Schema	4
2.3 Code Repository Link	4
2.4 Video Demo Link	4
3. Conclusion	5
3.1 Challenges and Solutions	5
3.2 Lessons Learned	5
3.3 Future Plan	5

1. Introduction

1.1 About the App

Famnest is a family finance and organizing app which includes expense tracking , document storing , events planning and preserving family's special moments.

1.2 Motivation

A few months ago, I faced a situation that made me realize the importance of having an organized system for family documents. During an emergency, I urgently needed a crucial family document, but despite searching everywhere, I couldn't find it in time. The frustration of not having easy access to something so important made me think—why isn't there a simple, secure way for families to store and retrieve essential documents when needed?

This experience led me to build **FamNest**, an app designed to help families keep their important files, financial records, event plans, and cherished memories all in one place. With **FamNest**, families can not only store documents securely but also manage expenses, plan events, and preserve special moments effortlessly. My goal is to make family management stress-free, so no one has to go through the inconvenience I once did.

-Md. Shahriar Kabir

1.3 Features

- **Authentications :**
 - **Signup :** Users can register for a account
 - Terms and conditions / Privacy policy : Must be accepted during signup via a checkbox.
 - **Login :** Users can securely log in to their accounts.
 - **Forget Password :** Users can reset their password by receiving a reset code. (Currently, the reset code is stored in the database, but OTP functionality will be implemented in the future.)
- **Multiple Groups Creation and Joining:** Users can create multiple groups as their needs and also can join any group with secured code.

- **HomePage:**The core of the application-
 - **Group Selection:** Users can choose a group to interact with.
 - **Navigation Drawer:** Provides access to key application features.
 - **Edit Profile:** Allows users to update their name, email, and profile picture.
 - **Groups:** Users can view the groups they have created and joined. Users also can leave the group and remove the member of the group.
 - **Change Password:** Users can update their account password.
 - **Logout:** Users can securely log out of the application.
 - **Create and join Group:** Users can create and join new groups anytime after logging in.

- **Main Features:** The application offers four core features, each designed to enhance usability and convenience:
 - **Expense Tracking :**Efficiently manages budgets and expenses across categories like medical, education, finance, and more, with detailed month-wise and year-wise analysis through intuitive graphs.
 - **Add budget :** .Users can set a budget for a specific date, with categories such as Groceries, Shopping, Education, Transport, and more.
 - **Add expense :** Users can set an expense for a specific date with categories.
 - **Month wise analysis and Year wise analysis :** Users can view your data with two graphs: one is line line graph and other is bar graph .
 - **Documents Storing :**Efficiently manage and organize important family documents with advanced features:
 - **Secure Storage:** Keep essential documents like medical records, legal papers, educational certificates, and financial statements safely stored.
 - **Custom Categories:** Users can create and organize documents into custom categories for better management with preset categories like Education, Medical, Finance .
 - **Nested Folders:** Create folders within folders to maintain a structured hierarchy for easy access.
 - **Flexible File Upload:** Upload files from any source, including device storage, Google Drive, and other cloud services.

- **File Management:**
 - Rename, delete categories, folders, and files as needed.
 - Organize recent documents efficiently for quick retrieval.
 - **Download & Share:**
 - Easily download files to your device.
 - Share documents with others securely when needed.
 - **View Documents & Media:**
 - Open documents directly within the google drive app.
 - View images, videos, and audio files seamlessly inside the application.
 - After downloading, files can be opened in any preferred app on the device.
- **Event Planning :** Create and organize events for specific dates with essential details such as event name, location, URL, and description, ensuring everything is easily accessible and well-planned. User also can edit and delete events.
- **Memories / Time Capsule:** this feature is designed to allow families to preserve special moments , memories and messages:
- **Multiple Categories:**
Family can store their precious moments in four forms : image, video , voice (audio) and in the form of stories
 - **Grid / Gallery View:**
The images and videos are arranged in the form of grid for proper user friendly environment
 - **Flexible File Upload:**
Users can easily upload files from their device and google drive.
 - **Smaller versions of images are shown in gallery view.**
 - **Audio Player and Video Player:**
Added audio and video player so that users can run the files smoothly on the app.
 - **Share and Download option:**
Users can easily share and download images , videos and voices.
 - **Mandatory available options :**
Options for editing stories and renaming the file names are available. Also there is a delete option for deleting files. **Searching** a file from chunks of file by file name is also there.
 - **Gradient Color:**
Used gradient color that gives an aesthetic appeal and is useful for engaging users.

1.4 Tools, Technologies and Frameworks Used

1. **FLUTTER** - Cross Platform mobile app development framework
2. **Android Studio** - IDE for building android apps and managing flutter projects
3. **FAST API** - Fast and Modern Python Framework for building API's
4. **CLOUDINARY** - Cloud Service for managing and optimizing images,videos,files
5. **MONGODB ATLAS** - Managed Cloud Database for scalable,flexible Data storage
6. **RENDER** - Platform for easy deployment of web apps and databases

1.5 Individual Responsibilities

1.5.1 Tawyabul Islam Tamim -04

Analysis Component of Expense Tracking

1. **Year-Wise Analysis**
 - **Line Graph:**
 - Displays a comparison between total yearly expenses and the yearly budget over the past 5 years.
 - **Bar Graph:**
 - Provides a category-wise comparison between expenses and budgets for each category, such as Groceries, Shopping, Education, etc.
2. **Month-Wise Analysis**
 - **Line Graph:**
 - Illustrates the monthly variation in expenses and budgets throughout the year.
 - **Bar Graph:**
 - Compares total expenses and budgets across various categories such as Groceries, Education, and Medical.

Key Features:

- **Dynamic Graph Updates:**
 - Real-time updates that reflect the latest expense and budget data for each month and category.
- **Line Graphs:**
 - A comparative view of monthly expenses versus budgets.
- **Bar Graphs:**
 - Provides category-wise insights into expenses and budgets.
- **Responsive Design:**
 - Ensures a balanced layout and consistent aspect ratio across various screen sizes.

Frontend Design:

- **Drawer Menu:**
 - Terms and Conditions
 - Privacy Policy
 - About
 - Log Out

1.5.2 Name: Md. Shahriar Kabir -20

Conceptualization

- **Core Idea Development:** Ideated and conceptualized the core idea of the app.

Activity Organization

- **Strategic Structuring:** Organized and streamlined all activities within the app for better functionality and user flow.

Frontend Design

- **User-Friendly Screens:** Designed and developed intuitive and visually appealing screens, including:
 - **Splash Screen:** Engaging app introduction with branding.
 - **Sign-Up Screen:** Simplified user registration process.
 - **Forgot Password Screen:** User-friendly password recovery interface.
 - **Main Homepage Screen:** Central dashboard for app navigation.
 - **Create Or Join Group Screen:** Options to create or join user groups.
 - **Groups Screen:** View and manage all groups in one place.
- **Document Store Module:**
 - **Document Store Screen:** Centralized document management.
 - **Category Page Screen:** Organized document categories.
 - **Folder Page Screen:** Individual folder view.
 - **Nested Folder Page Screen:** Detailed sub-folder navigation.
- **Expense Tracking Module:**
 - **Expense Tracking Screen:** Overview of all expenses.
 - **Expense Screen:** Manage and log expenses.
 - **Budget Screen:** Set and track budgets.
- **Plan Events Module:**
 - **Event Planning Screen:** Tools for organizing and tracking events.
- **Memories Module:**

- **Time Capsule Screen:** Store and revisit cherished memories.

Key Features Implementation

- **Nested Sub-Folders:** Developed a multi-level document storage system to improve organization.
- **Multiple Groups Mechanism:** Enabled users to create, join, and manage multiple groups effortlessly.

State Management

- **Enhanced User Experience:** Implemented robust state management to ensure a seamless and efficient user experience across the app.

Modular Project Folder Structure

- Created a **scalable and maintainable project structure** with feature-based modules (e.g., **authentication, features, backend services**).
- Each module contains components organized into distinct folders for **clarity, efficiency, and scalability**.

Database Design and Implementation

- **Optimized Databases:** Designed the whole database structure and implemented databases to support:
 - **Event Plans:** Storing and managing event details.
 - **Document Storing:** Secure and organized document storage.

Backend Development

- **Critical Feature Implementation:** Developed backend functionality for key features:
 - **Secured User Password:** Implemented password hashing for security.
 - **Forgot Password Mechanism:** Password recovery with hashing to maintain data security.
 - **Multiple Groups Functionality:** Backend logic for creating and managing groups.
 - **Groups Screen:** Implemented backend logic for fetching groups.
 - **Document Storing System:** Efficient backend support for document management.
 - **Event Planning Module:** Integrated backend for planning and managing events.

Responsibilities:

- **Frontend Development:**
 - Shaped the app's core concept for smooth user interaction.
 - Selected Flutter widgets to improve UI and user experience.
 - Implemented text field validation for email, password, and group names.
- **Testing & Debugging:**
 - Conducted testing and fixed UI bugs for responsiveness.
 - Optimized frontend code for better stability and performance.
- **Backend Development:**
 - Designed an efficient MongoDB database for data storage and retrieval.
 - Addressed duplicate group names by implementing unique group codes.
 - Enabled multiple group management with unique codes.
 - Developed backend logic for expense tracking (adding, retrieving, categorizing).
- **Cloudinary Integration:**
 - Tested Cloudinary's integration with FastAPI for media handling.
 - Supported teammates with Cloudinary implementation.
- **Legal Documentation:**
 - Led the creation of Terms & Conditions and Privacy Policy to ensure legal compliance.
 - Ensured transparency in data handling practices for users.
- **Collaboration & Teamwork:**
 - Worked closely with the team to integrate frontend and backend features.
 - Promoted effective communication for idea sharing and issue resolution.

1.5.4. Faiaz Mahmud Ifti - 46

- **Fundamentals:**

- Helped team members in implementing some of the frontend pages at the initial stage and along the way.
- Organized the backend part of our project and established connection of our app with backend and cloud storage.
- **FastAPI Connection with App:**
Connected the flutter project app to the Database MongoDB atlas using python fastAPI as backend
- **Backend Connection with front end pages:**
Made the fundamental backend connections of **login page, sign up page, create join group page , change password , edit profile** etc. and stored them in a database. Also implemented the corresponding Snackbar messages and alert messages for them.
- **Using hosting Service to make database and fastAPI global:**
At first I made the backend connections on my local machine for testing. Then for global reachout I looked out for options and chose Render as the online host service and uploaded the main.py file on github then connected the render to github.
- **Connected database and fastAPI to Cloud Storage:**
We used Cloudinary as the Cloud Storage for storing the pictures and videos. I connected the Cloudinary credentials to our main.py.

- **Special Features Implemented :**

- **TIME CAPSULE Implementation:**
 - Implemented one of the four main features of our App - “Time Capsule”
 - Divided it into four categories -> **images , videos , audios and stories**
 - **Storing URLs to MongoDB from Cloud:**

connected the database to flutter for uploading the pictures, videos, stories, audios and stored the Cloudinary url of them in there. Also implemented delete function to remove image or video from the database as well as from the cloud.
 - **Search Bar , Upload Option , Edit/Rename and Delete Options** are available.

- View **Full Screen Mode** : Added full view for images, videos , audios and stories.
- Dynamic **Gradient Color**: Used Color Gradient to make the screens attractive.
- **Audio Player and Video Player** for running the file in the app
- Provided **Grid View** to Image and Videos section for gallery view
- **Download and Share Button**:

Added **Download button and Share button** in the three sections - image, video and audio.

- **Side Features:**

- **EDIT PROFILE page :**

- Made the Edit Profile functionality (i.e name , email and update profile picture) and set an image as the default profile picture

- **Change User Password:**

- Implemented the Change Password feature for user

- **Password Generation for Groups:**

- Created auto generated strong passwords for each Group. The length is also variable (13 to 15 characters) to strengthen security.

- **INPUT VALIDATION:**

- Added Validator to check the correctness/validity of essential parameters like name , email, group name criteria etc.

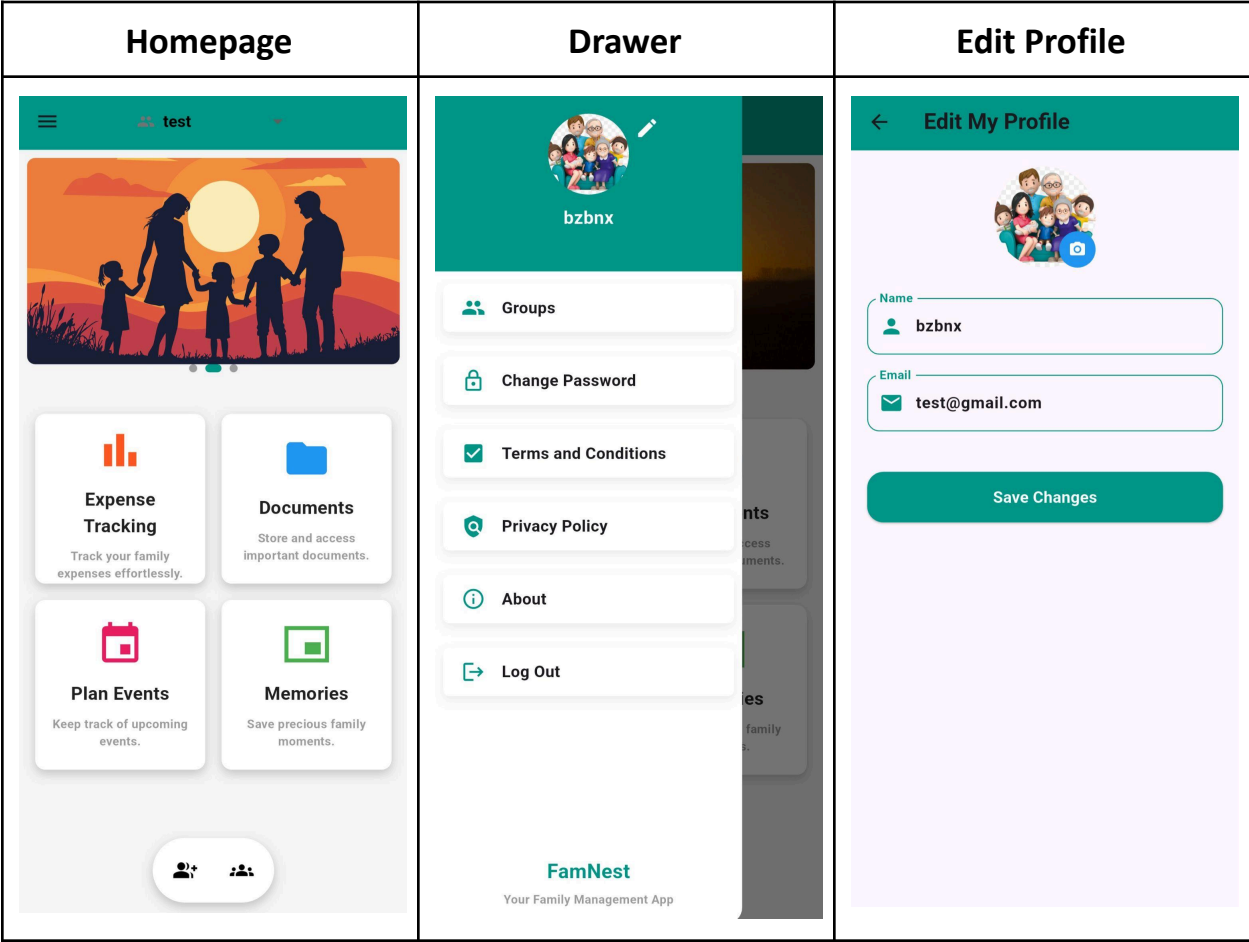
2. Design and Implementation

2.1 UI Developed

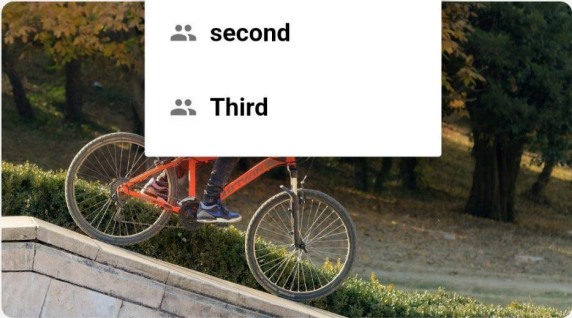
Splash Screen	Login page	SignUp Page
 FamNest <i>Your Family's Digital Hub</i>	 Welcome Back Log in to your FamNest account.  Email  Password  Forgot Password? Login Don't have an account? Sign Up	 Create Your Account Join FamNest and manage your family's life in one place!  Name  Email  Password   Confirm Password  ☐ I have read and agree to the Terms of Services and Privacy Policy. Sign Up Already have an account? Login

Terms Of Services	Forgot Password	Forgot Password
 Terms of Services **FamNest Terms of Service** **Last Updated: December 10, 2024** Welcome to FamNest! By accessing or using our mobile application, you agree to comply with and be bound by the following Terms of Service. Please read these terms carefully. If you do not agree, please refrain from using our services. --- **1. Acceptance of Terms** By creating an account or using FamNest, you agree to abide by these Terms of Service and our Privacy Policy. These terms apply to all users, including family members who share access. --- **2. Use of Services** FamNest provides tools for families to: - Manage and track financial expenses. - Store and access important documents. - Manage family events via a shared calendar. - Store family memories such as photos, videos, and Close	←Forgot Password Email Send Reset Code	←Forgot Password Reset Code <> New Password Reset Password

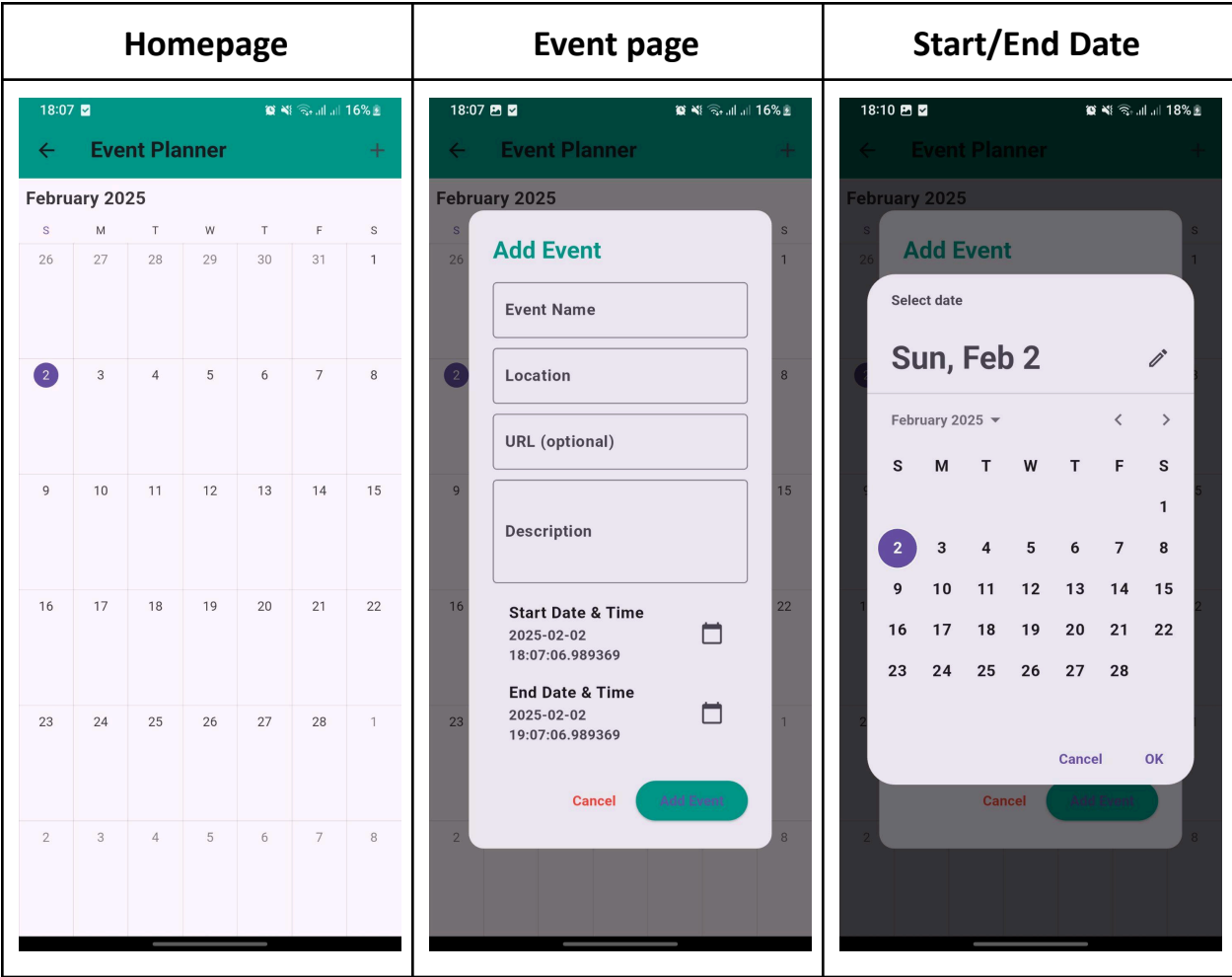
Group Name	Group Code	Group Password
 Create or Join Your First Group  Group Name Have a code? Join Group Cancel Done Login Successful.	 Create or Join Your First Group Enter Group Code  Group Code Want to create a new group? Create Group Cancel Done	 Create or Join Your First Group Group Password Your Group Password is: KkWNiFjIKvqeuW OK Created Group Successfully

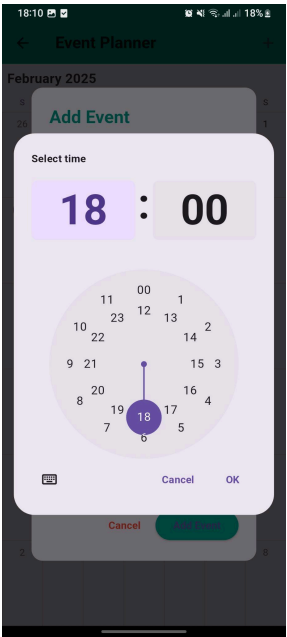
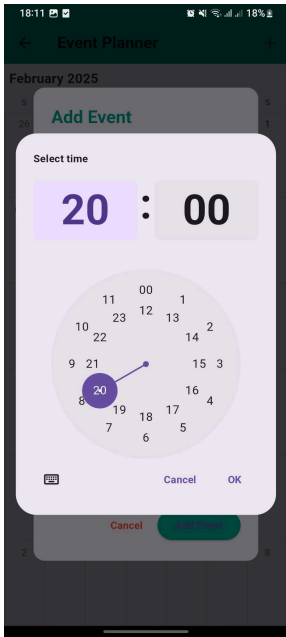
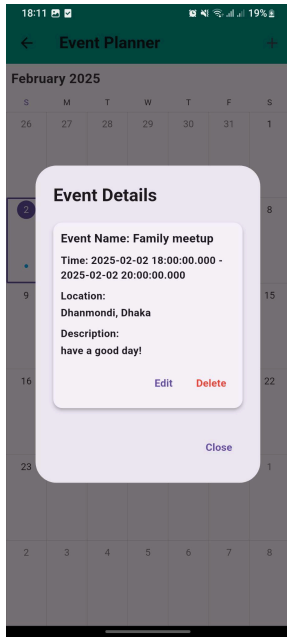
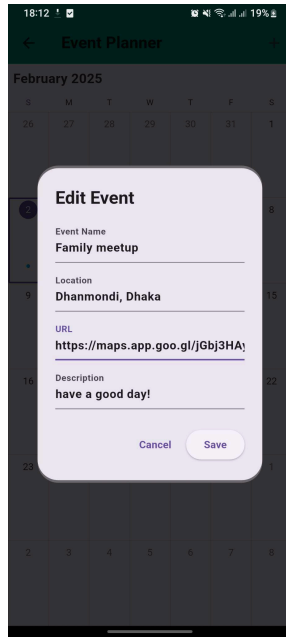


Change Password	Terms and Conditions	About
← Change Password 🔒 Current Password 👁 🔒 New Password 👁 🔒 Confirm New Password 👁 Change Password	← Terms And Conditions Terms and Conditions Welcome to FemNest, your family's go-to platform for managing important moments, finances, and memories. These Terms and Conditions ("Terms") govern your access and use of the FemNest app and services. By using FemNest, you agree to comply with these Terms. Please read them carefully before proceeding. If you disagree with any part of the Terms, discontinue using the app. 1. Acceptance of Terms By creating an account or using FemNest, you agree to abide by these Terms and our Privacy Policy. These Terms apply to all users, including individual family members who share access to the platform. 2. Services Provided FemNest offers features designed to help families stay organized and connected: - Securely store and access important family documents. - Manage and monitor family finances. - Preserve memories like photos, videos, and	← About  Welcome to Famnest! Team Name : Team Extreme Tawyabul Islam Tamim - 04 Md. Shahriar Kabir - 20 Ovijit Chandra Balo - 28 Faiaz Mahmud Ifti - 46

Groups In Homepage	Groups Info
test second Third  Expense Tracking Track your family expenses effortlessly. Documents Store and access important documents. Plan Events Keep track of upcoming events. Memories Save precious family moments.	Groups test Code: KkWNiFjIKvqeuW second Code: cpn8k9bP7;j+RaGd Third Code: cTig2P(?Pksh?_ Group: Third Leave Group test@gmail.com Joined: 2025-02-02T11:17:39.303000

Event Planning :



Start Time	End Time	Event Details	Edit Event
			

Expense Tracking :

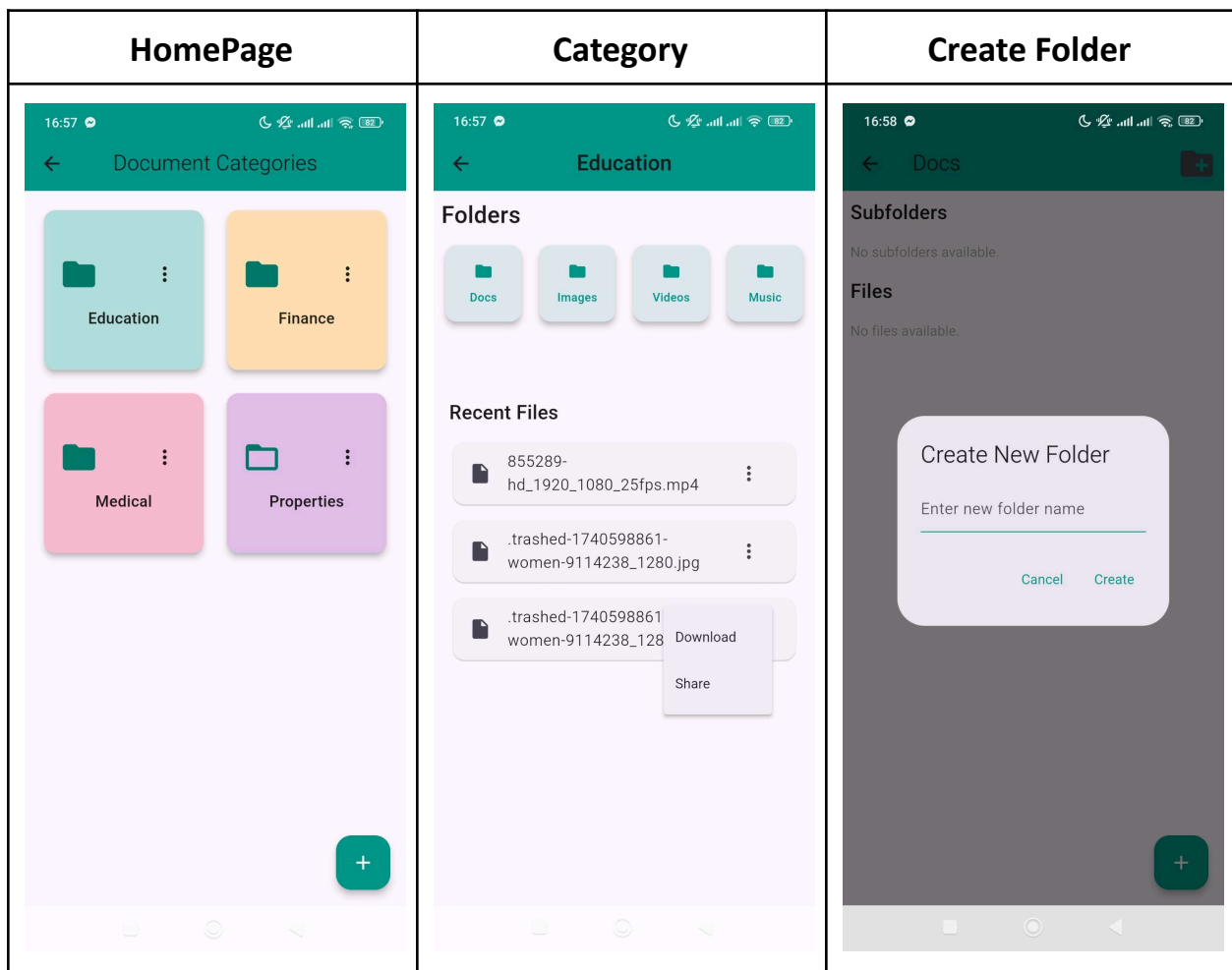
Homepage	Create Budget	Category List
18:25 Expense T... February 2025 Budget 0.00 100% Expense 0.00 0% Remaining 0.00 0% Expenses Overview No budgets available.	18:25 Create New Budget Select a Month Category Education Amount Save Budget	18:25 Create New Budget Groceries Shopping Education Transport Bills Medical Others

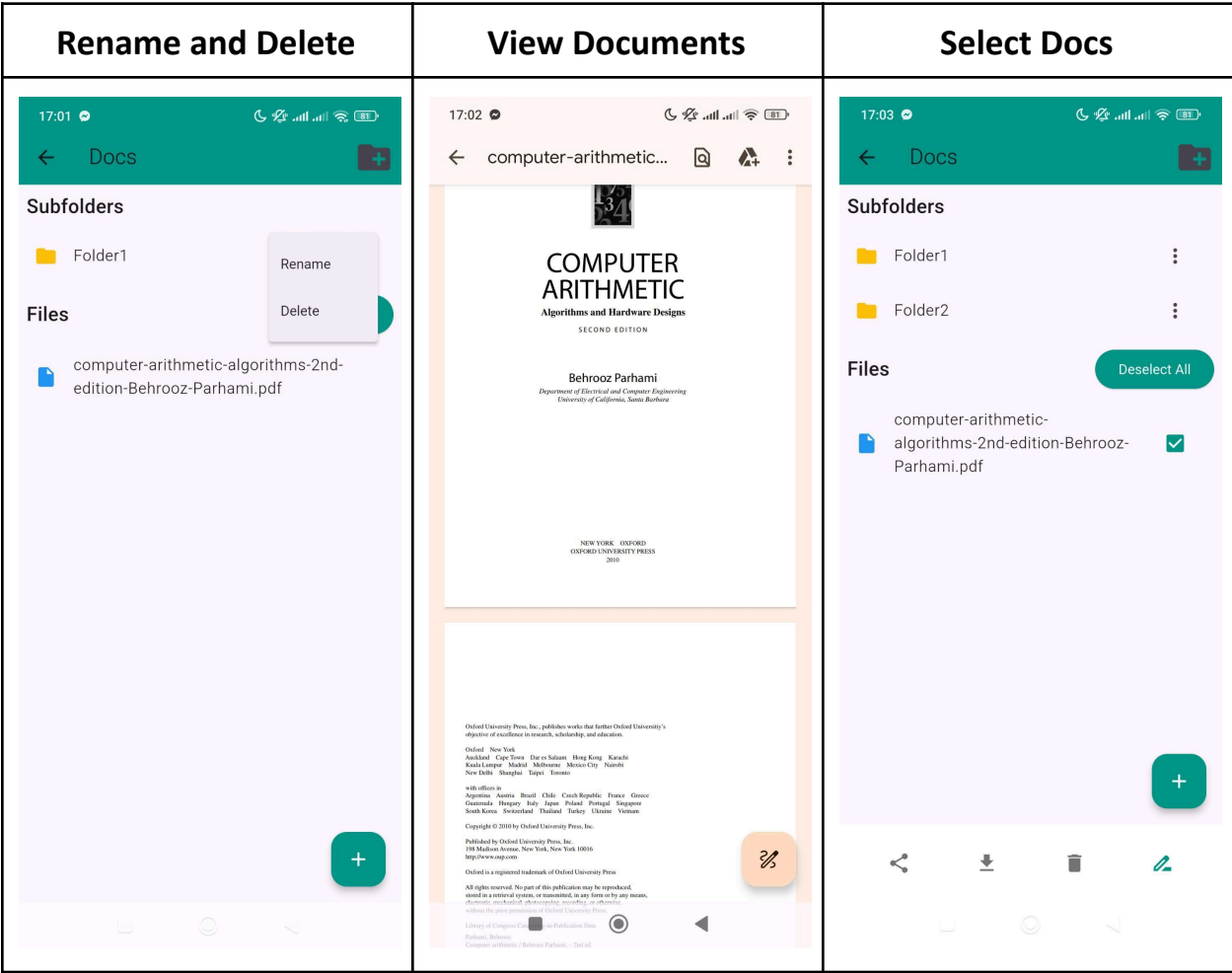
The image displays three side-by-side mobile application screens, each with a dark green header bar and a light purple background.

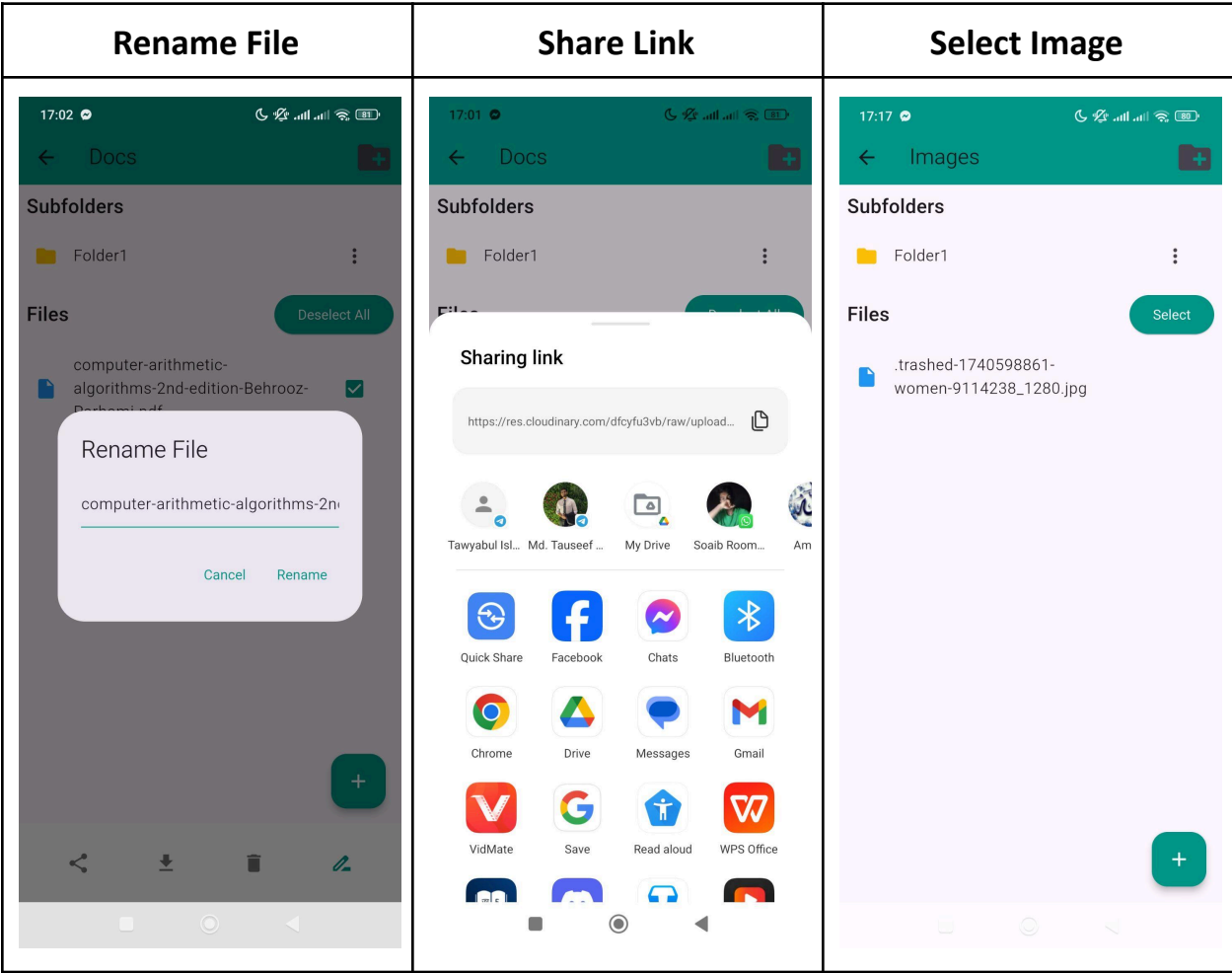
- Budget List:** The header bar contains a back arrow, the title "Budgets", and a plus icon. Below the header, the title "Your Budgets" is followed by a date selector "February 2025" with a calendar icon. Three budget items are listed in light gray rounded rectangles:
 - Education:** Amount: ₹1000.00
 - Transport:** Amount: ₹200.00
 - Medical:** Amount: ₹500.00
- Add Expense:** The header bar contains a back arrow, the title "Add Expense", and a plus icon. The form includes:
 - A date input field showing "2025-2-2" with a calendar icon.
 - A category dropdown menu currently set to "Education".
 - An "Amount" input field.
 - A rounded "Add Expense" button.
- Expense List:** The header bar contains a back arrow, the title "Expenses", and a plus icon. Below the header, the title "Expenses for 2/2025" is followed by a date selector "2/2025" with a calendar icon. Three expense items are listed in light yellow rounded rectangles:
 - February 2, 2025:** Education, ₹500.00
 - February 2, 2025:** Transport, ₹700.00
 - February 2, 2025:** Medical, ₹400.00

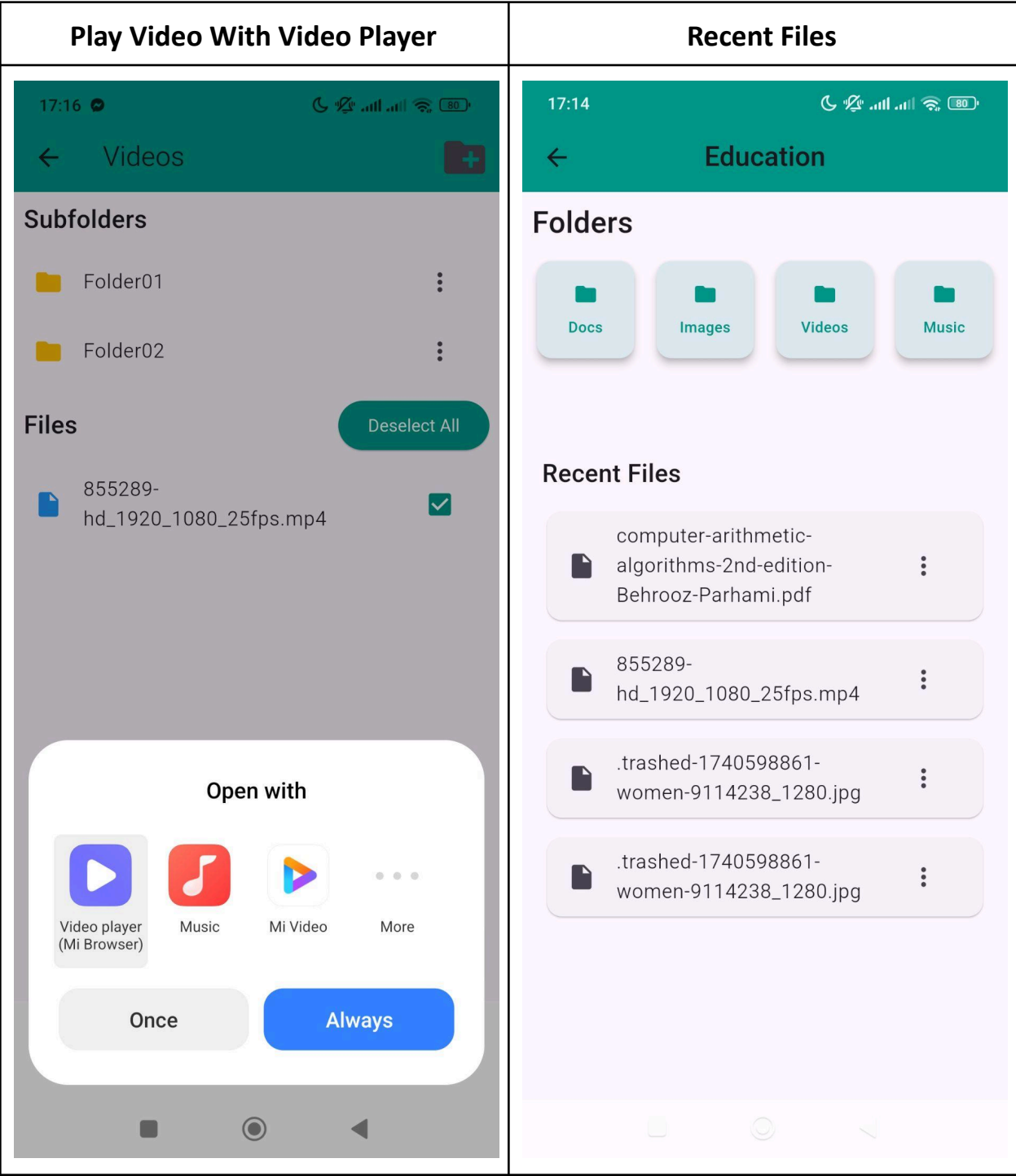


Documents Storing :

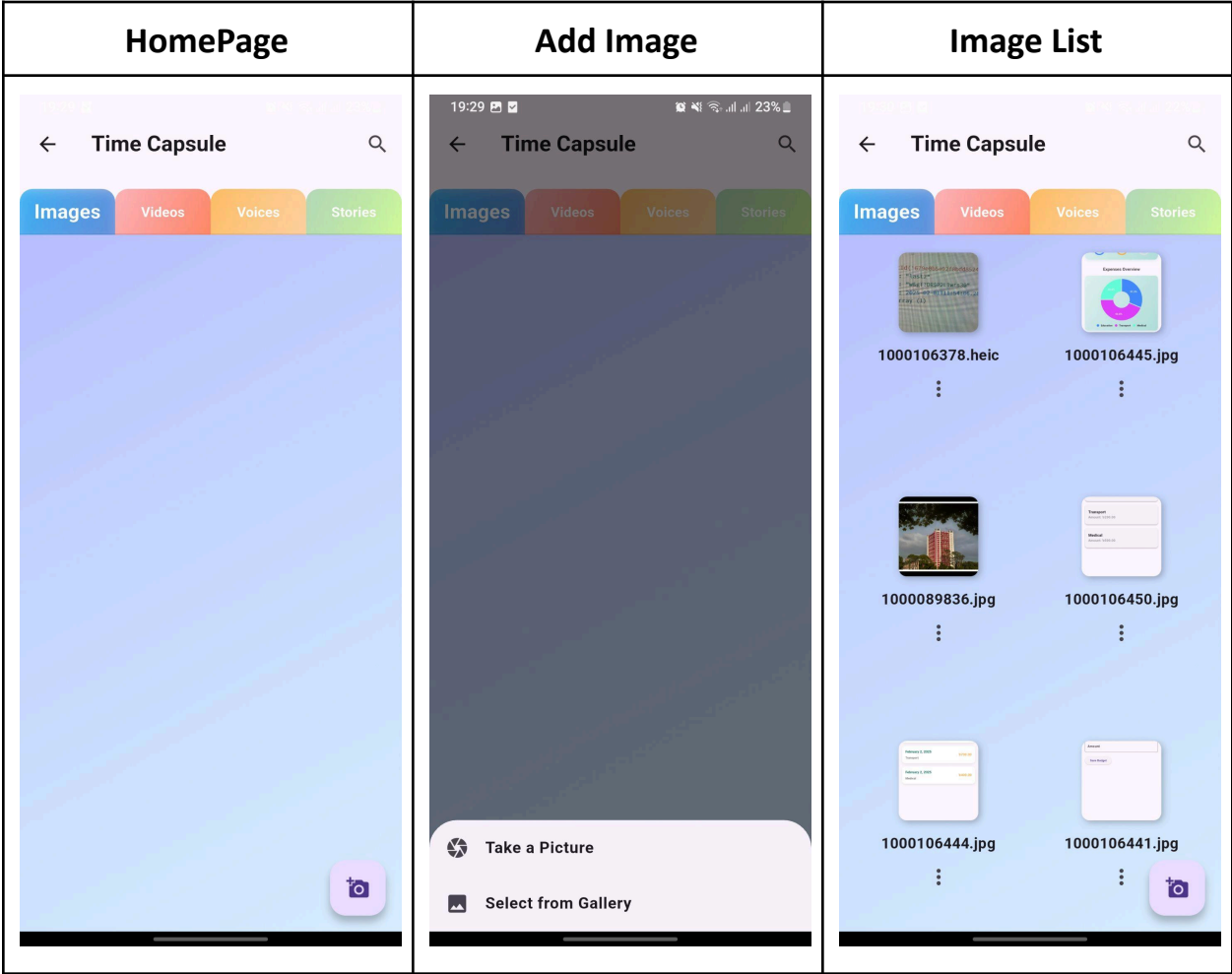


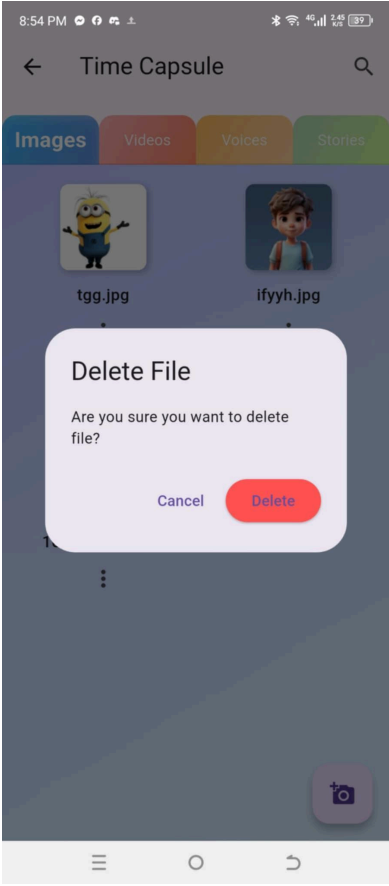


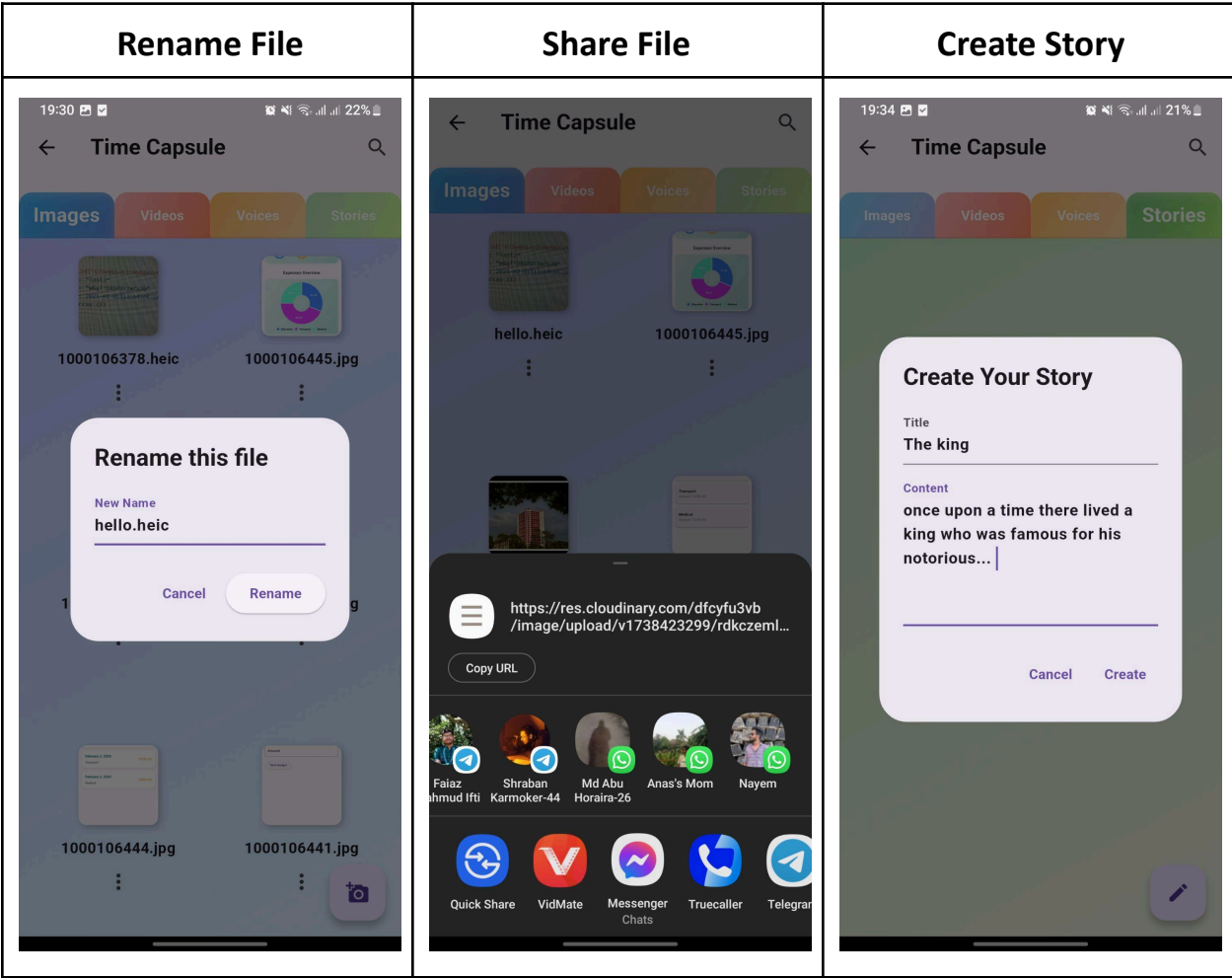


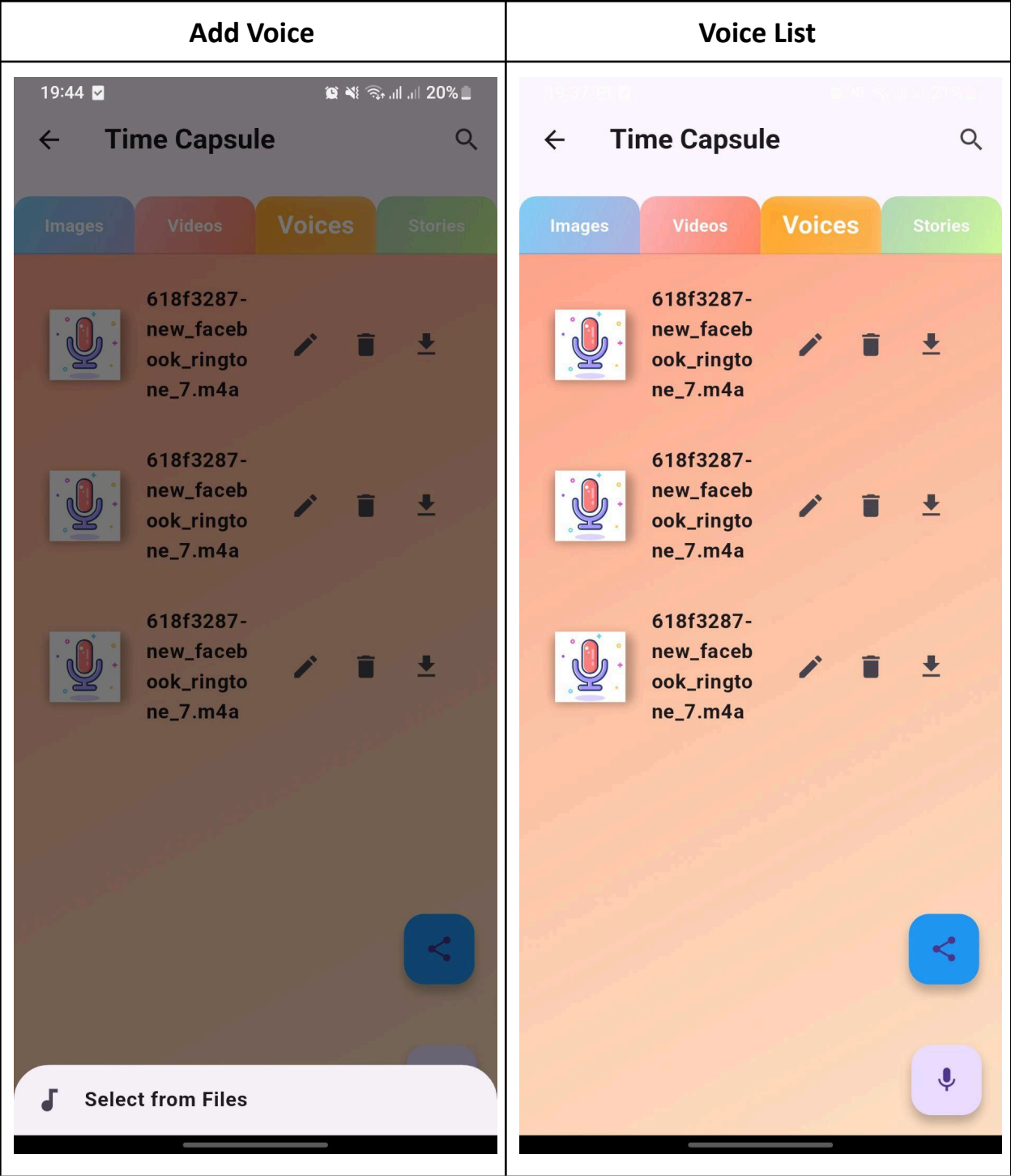


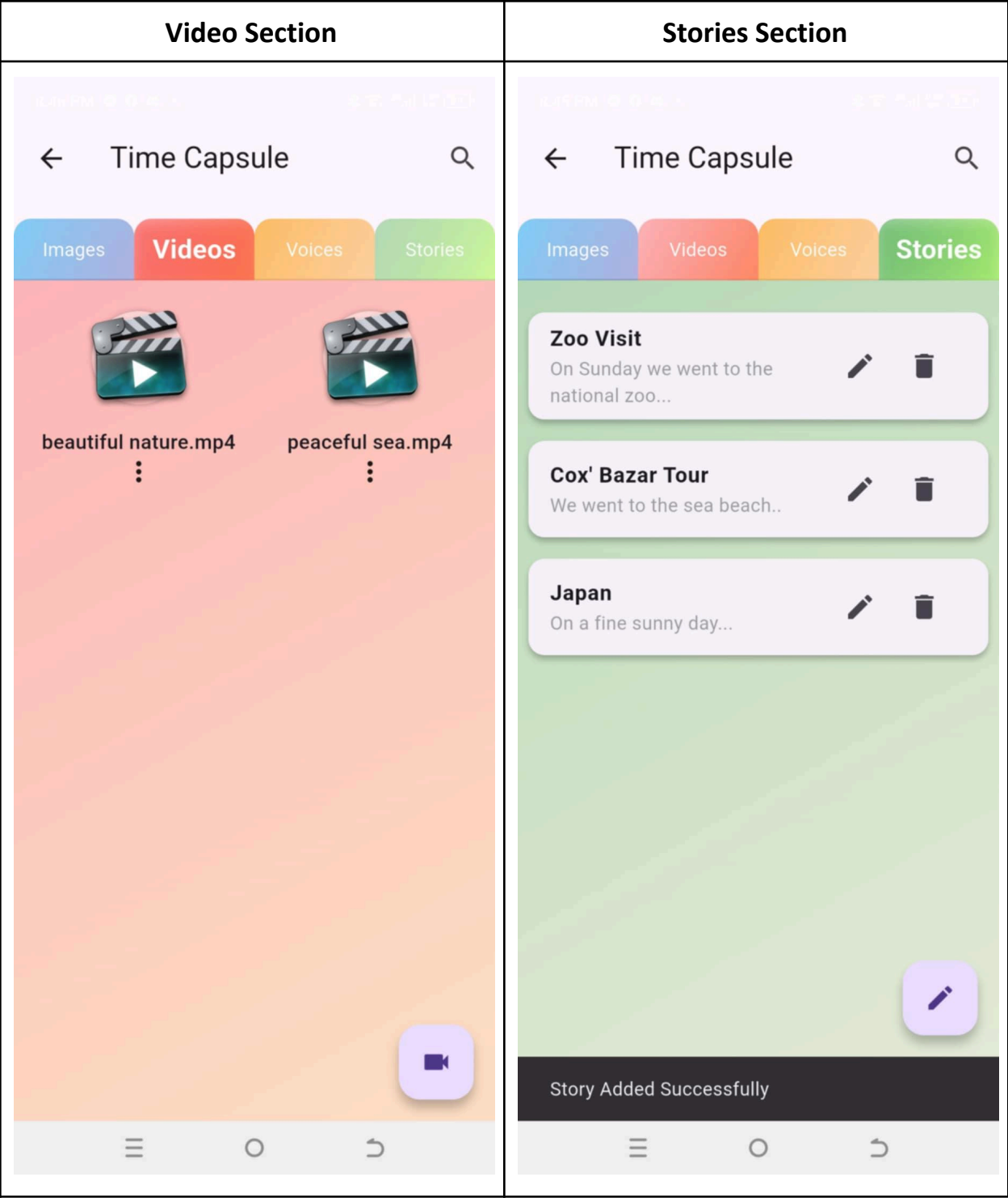
Memories / Time Capsule:



Options	Delete File confirmation	Deleted File
 The screenshot shows the 'Time Capsule' app interface. At the top, there's a status bar with the time 8:55 PM and battery level at 39%. Below the app title 'Time Capsule', there are four tabs: 'Images' (selected), 'Videos', 'Voices', and 'Stories'. Two image thumbnails are visible: 'tgg.jpg' (a Minion) and 'ifyyh.jpg' (a cartoon boy). A context menu is open at the bottom, listing four options: 'Rename' (with a pencil icon), 'Delete' (with a trash can icon), 'Download' (with a download icon), and 'Share' (with a share icon). The bottom navigation bar shows three icons: a menu, a home circle, and a back arrow.	 This screenshot shows the same 'Time Capsule' app interface as the first panel, but with a confirmation dialog box in the center. The dialog is titled 'Delete File' and asks 'Are you sure you want to delete file?'. It has two buttons: 'Cancel' and 'Delete' (which is highlighted in red). The background is dimmed, and the status bar shows the time as 8:54 PM.	 The screenshot shows the 'Time Capsule' app after the file deletion. A green banner at the bottom of the screen displays the message 'File Deleted Successfully'. The image thumbnails 'tgg.jpg' and 'ifyyh.jpg' are still visible, but the context menu is no longer present. The status bar at the top shows the time as 8:55 PM.







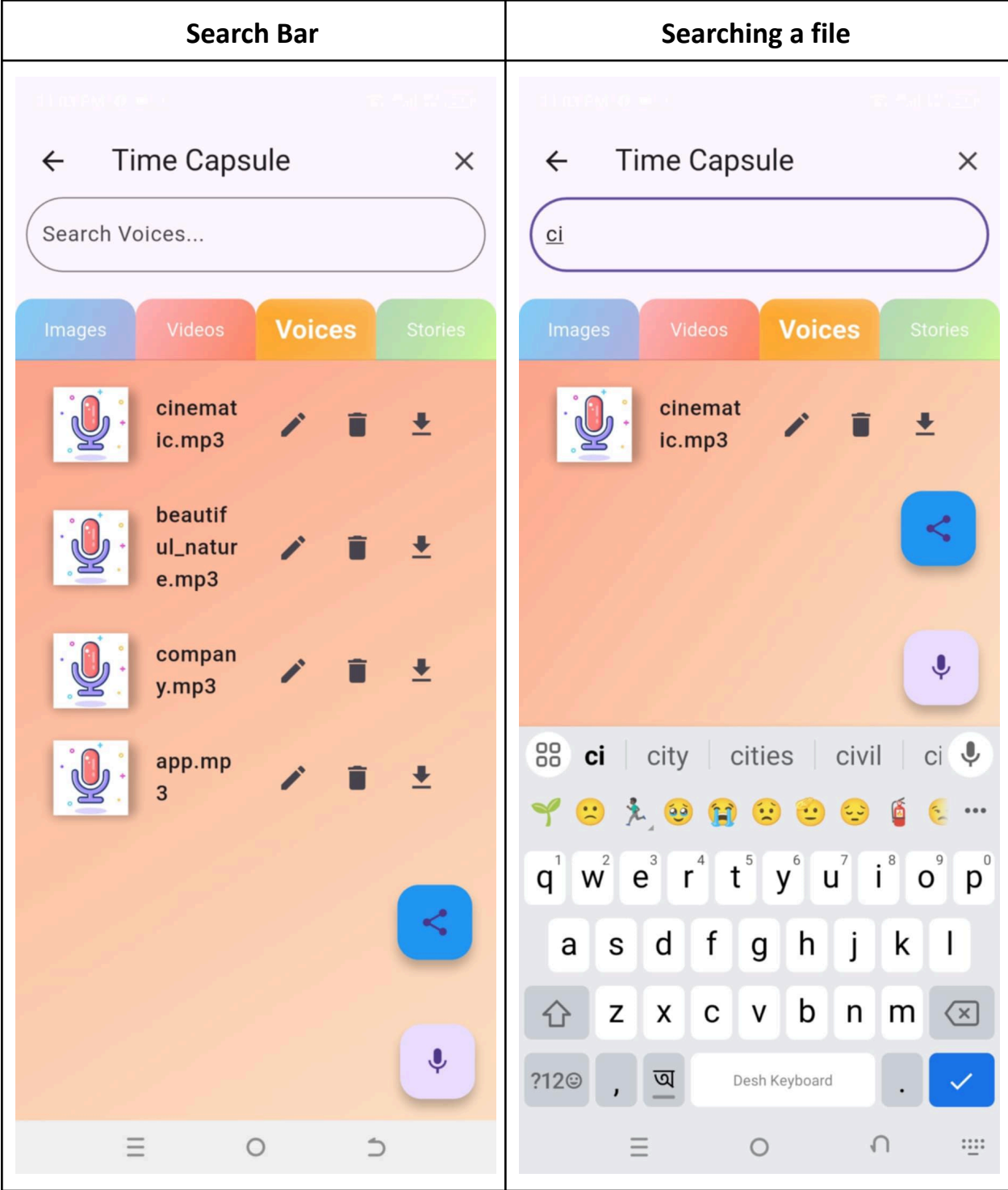
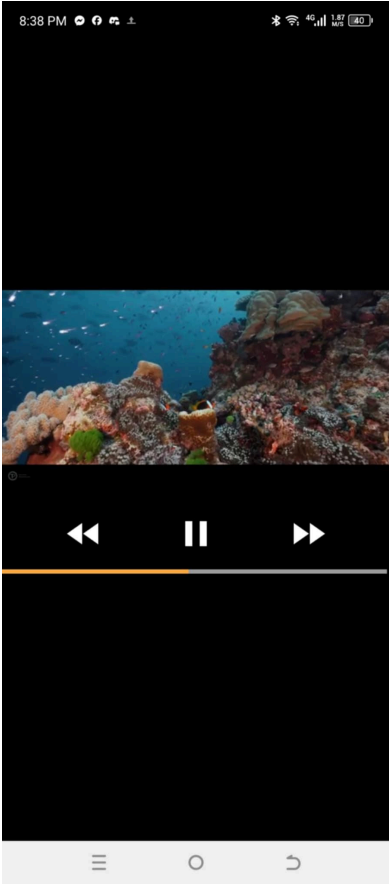
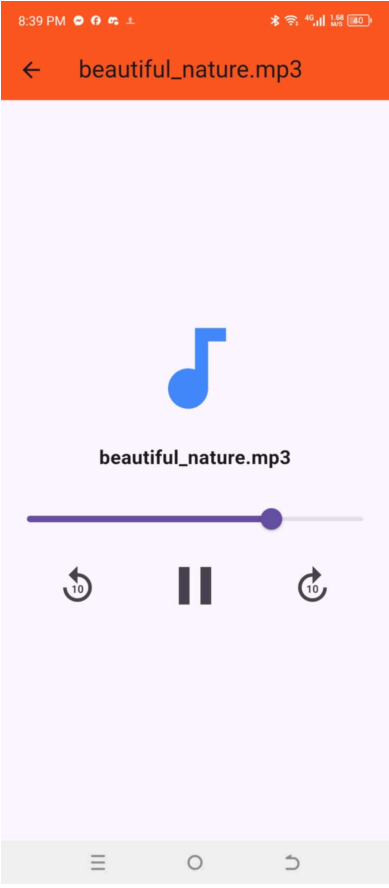


Image View	Video View	Audio View
 A mobile app mockup for the 'Image View' screen. The status bar at the top shows the time as 8:39 PM, signal strength, and battery level. The main content area displays a portrait of a man with glasses and a beard. The bottom navigation bar contains three icons: a hamburger menu, a circle, and a right-pointing arrow.	 A mobile app mockup for the 'Video View' screen. The status bar shows the time as 8:38 PM. The main content area shows a video of a coral reef. Below the video is a playback control bar with a progress line and three icons: a double left arrow, a pause icon, and a double right arrow. The bottom navigation bar is identical to the Image View screen.	 A mobile app mockup for the 'Audio View' screen. The status bar shows the time as 8:39 PM. The header bar is orange and contains a back arrow and the filename 'beautiful_nature.mp3'. The main content area features a large blue musical note icon, the filename 'beautiful_nature.mp3', a progress slider, and playback controls including a 10-second rewind icon, a pause icon, and a 10-second play icon. The bottom navigation bar is identical to the other screens.

2.2 Database Schema

+ Create Database

Q Search Namespaces

FamNest

Categories
Events
Files
Folders
Groups
RecentFiles
TimeCapsuleMediaFiles
Users
budget
expense
sample_mflix

FamNest

LOGICAL DATA SIZE: 72.22KB STORAGE SIZE: 372KB INDEX SIZE: 364KB TOTAL COLLECTIONS: 10

CREATE COLLECTION

Collection Name	Documents	Logical Data Size	Avg Document Size	Storage Size	Indexes	Index Size	Avg Index Size
Categories	72	6.62KB	95B	36KB	1	36KB	36KB
Events	17	3.95KB	238B	36KB	1	36KB	36KB
Files	17	5.69KB	343B	36KB	1	36KB	36KB
Folders	308	38.03KB	127B	48KB	1	40KB	40KB
Groups	23	3.73KB	167B	36KB	1	36KB	36KB
RecentFiles	12	3.94KB	337B	36KB	1	36KB	36KB
TimeCapsuleMediaFiles	9	2.65KB	302B	36KB	1	36KB	36KB
Users	8	4.52KB	579B	36KB	1	36KB	36KB
budget	12	1.43KB	122B	36KB	1	36KB	36KB
expense	16	1.65KB	106B	36KB	1	36KB	36KB

Fig:DB Collections

```
_id: ObjectId('679e3b1991ba7940d6cd0230')
category_name: "Education"
group_code: "?|mp}qnmDC]cn{"
is_preset: true
```

```
_id: ObjectId('679e3b1a91ba7940d6cd0235')
category_name: "Finance"
group_code: "?|mp}qnmDC]cn{"
is_preset: true
```

Fig:Instances of Categories

```
_id: ObjectId('679e3caa91ba7940d6cd0266')
group_code: "2Ny0GL+<59n$%MQ"
name: "Testing"
start_date: 2025-02-01T21:24:14.195+00:00
end_date: 2025-02-01T22:24:14.195+00:00
location: "dhaka"
url: ""
description: "dhaka"
created_at: 2025-02-01T15:24:26.710+00:00

_id: ObjectId('679e59c6c832ca28fd01b483')
group_code: "vaBCm0AvWPfH009H"
name: "CSE du fresher's week"
start_date: 2025-02-01T23:28:09.390+00:00
end_date: 2025-02-02T00:28:09.390+00:00
location: "CSE dept"
url: "https://maps.app.goo.gl/ZPf5Vr9vRe5fdPJL9"
description: "fun, fun and fun"
created_at: 2025-02-01T17:28:38.415+00:00
updated_at: 2025-02-01T18:08:29.526+00:00
```

Fig:Instances of Events

```
_id: ObjectId('679e3baa91ba7940d6cd0241')
file_name : "Lecture 18 - Rod Cutting and Rock Climbing.pdf"
category_id : "679e3b1991ba7940d6cd0230"
folder_id : "679e3b1991ba7940d6cd0232"
cloudinary_url : "https://res.cloudinary.com/dfcyfu3vb/raw/upload/v1738423210/ewuny5vj1c..."
public_id : "ewuny5vj1c8rkqkc9q3d"
file_type : "raw"
created_at : 2025-02-01T15:20:10.586+00:00
```

```
_id: ObjectId('679e53c0c832ca28fd01b45c')
file_name : ".trashed-1740598861-women-9114238_1280.jpg"
category_id : "679e5391c832ca28fd01b44d"
folder_id : "679e5391c832ca28fd01b44f"
cloudinary_url : "https://res.cloudinary.com/dfcyfu3vb/image/upload/v1738429375/pluw3dry..."
public_id : "pluw3dryowdkgjnxdwmo"
file_type : "image"
created_at : 2025-02-01T17:02:56.319+00:00
```

Fig:Instances of Files

```
_id: ObjectId('679e3b1991ba7940d6cd0231')
folder_name : "Docs"
category_id : "679e3b1991ba7940d6cd0230"
parent_folder_id : null
created_at : 2025-02-01T15:17:45.512+00:00
```

```
_id: ObjectId('679e3b1991ba7940d6cd0232')
folder_name : "Images"
category_id : "679e3b1991ba7940d6cd0230"
parent_folder_id : null
created_at : 2025-02-01T15:17:45.741+00:00
```

Fig:Instances of Folders

<pre> _id: ObjectId('679f3ad2c29f6d7fb6e3e646') group_name : "hello" group_code : "WBXeSHBKOWRSJt2" created_at : 2025-02-02T09:28:50.090+00:00 ▼ members : Array (1) ▼ 0: Object email : "faiaz@gmail.com" joined_at : 2025-02-02T09:28:50.090+00:00 </pre>
<pre> _id: ObjectId('679f442622c0d6e29d423cf5') group_name : "hehi" group_code : "nYguHo9vCzBMewc" created_at : 2025-02-02T10:08:38.000+00:00 ▼ members : Array (2) ▼ 0: Object email : "faiaz@gmail.com" joined_at : 2025-02-02T10:08:38.000+00:00 ▼ 1: Object email : "iamovijit1792@gmail.com" joined_at : 2025-02-02T10:14:03.317+00:00 </pre>

Fig:Instances of Groups

<pre> _id: ObjectId('679e3baf91ba7940d6cd0242') group_code : "? mp}qnmDC]cn{" category_id : "679e3b1991ba7940d6cd0230" file_id : "679e3baa91ba7940d6cd0241" file_name : "Lecture 18 - Rod Cutting and Rock Climbing.pdf" file_type : "raw" cloudinary_url : "https://res.cloudinary.com/dfcyfu3vb/raw/upload/v1738423210/ewuny5vj1c..." last_accessed : 2025-02-01T15:20:15.804+00:00 </pre>
<pre> _id: ObjectId('679e53c8c832ca28fd01b45d') group_code : "pNuurkqNBYeqsZ" category_id : "679e5391c832ca28fd01b44d" file_id : "679e53c0c832ca28fd01b45c" file_name : ".trashed-1740598861-women-9114238_1280.jpg" file_type : "image" cloudinary_url : "https://res.cloudinary.com/dfcyfu3vb/image/upload/v1738429375/pluw3dry..." last_accessed : 2025-02-01T17:03:23.935+00:00 </pre>

Fig:Instances of RecentFiles

```
_id: ObjectId('679e3bc791ba7940d6cd0244')
group_code : "?|mp}qnmDC]cn{"
▼ uploaded_images : Array (1)
  ▼ 0: Object
    file_name : "1000355474.jpg"
    url : "https://res.cloudinary.com/dfcyfu3vb/image/upload/v1738423238/qmn9ejnr..."
```

```
_id: ObjectId('679e3c0491ba7940d6cd0255')
group_code : "LXk6v7WS}yb%SwV"
▼ uploaded_images : Array (1)
  ▼ 0: Object
    file_name : "1000106378.heic"
    url : "https://res.cloudinary.com/dfcyfu3vb/image/upload/v1738423299/rdkczeml..."
```

```
_id: ObjectId('679f6d4b7fddf6563c84f48e')
group_code : "nUfVe3izclQBzv3"
▼ uploaded_images : Array (1)
  ▼ 0: Object
    file_name : "istockimages1000355542.jpg"
    url : "https://res.cloudinary.com/dfcyfu3vb/image/upload/v1738501450/hczy6isr_"
▼ uploaded_videos : Array (1)
  ▼ 0: Object
    file_name : "1000355486.mp4"
    url : "https://res.cloudinary.com/dfcyfu3vb/video/upload/v1738501489/cfst4zb_"
▼ uploaded_audios : Array (1)
  ▼ 0: Object
    file_name : "01.hold my hand(duet with akon).mp3"
    url : "https://res.cloudinary.com/dfcyfu3vb/video/upload/v1738501510/nq8wqksc_"
▼ uploaded_stories : Array (1)
  ▼ 0: Object
    title : "test"
    content : "once upon a time there lived a king in Madrid"
```

```
_id: ObjectId('679e3c8491ba7940d6cd0255')
group_code : "LXk6v7WS}yb&SwV"
▸ uploaded_images : Array (8)
  ▸ 0: Object
  ▸ 1: Object
  ▸ 2: Object
  ▸ 3: Object
  ▸ 4: Object
  ▸ 5: Object
  ▸ 6: Object
  ▸ 7: Object
▸ uploaded_videos : Array (1)
  ▸ 0: Object
▸ uploaded_stories : Array (3)
  ▸ 0: Object
  ▸ 1: Object
  ▸ 2: Object
▸ uploaded_audios : Array (3)
  ▸ 0: Object
  ▸ 1: Object
  ▸ 2: Object
```

Fig: Instances Of Time Capsule Media files

```

_id: ObjectId('679e3aef91ba7940d6cd022d')
name: "iamovijit1792"
email: "iamovijit1792@gmail.com"
password: "$2b$12$VpdxXiGlCxbHnn1MsebaJedRlFI/H0rKjyVVaU2eKZiPkvqkTTF46"
▼ groups: Array (3)
  ▼ 0: Object
    group_name: "new_group"
    group_code: "2Ny0GL+<59n$%MQ"
  ▼ 1: Object
    group_name: "latest"
    group_code: "R0ZilULfERbgRR8"
  ▼ 2: Object
    group_name: "hehi"
    group_code: "nYguHo9vCzBMewc"
    created_at: 2025-02-02T10:08:38.000+00:00
▶ current_group: Object
profile_picture_url: "http://res.cloudinary.com/dfcyfu3vb/image/upload/v1738494042/profile_p..."
login_status: true
created_at: 2025-02-01T15:17:01.682+00:00
last_login: 2025-02-02T10:59:32.953+00:00
reset_code: "BBYAoJ"
reset_code_expiry: 2025-02-02T11:08:46.169+00:00

```

Fig: Instances Of Users

```
_id: ObjectId('679e3b4991ba7940d6cd023f')
category: "Education"
month: 2025-02-01T00:00:00.000+00:00
amount: 200
spent: 100
groupCode: "?|mp}qnmDC]cn{"
```

```
_id: ObjectId('679e6a54d617434a184145a6')
category: "Transport"
month: 2025-02-12T00:00:00.000+00:00
amount: 8866
spent: 0
groupCode: "GBX0p887jBhalwG"
```

Fig: Instances Of Budget

```
_id: ObjectId('679e3b5391ba7940d6cd0240')
category: "Education"
date: 2025-02-01T21:18:38.549+00:00
amount: 100
groupCode: "?|mp}qnmDC]cn{"
```

```
_id: ObjectId('679e6a6ed617434a184145a7')
category: "Groceries"
date: 2025-02-19T00:00:00.000+00:00
amount: 3580
groupCode: "GBX0p887jBhalwG"
```

Fig: Instances Of Expense

2.3 Code Repository Link

GitHub Repository: [FamNest](#)

2.4 Video Demo Link

📺 Screenrecorder-2025-02-02-18-55-16-77~2.mp4

3. Conclusion

3.1 Challenges and Solution

3.1.1 Challenges Arising from Gradle and JDK Version Incompatibilities:

Issue:

During the development process, we encountered a recurring issue where the Android Studio emulator unexpectedly ceased functioning. Despite exploring various troubleshooting methods, we were unable to identify a viable solution. As an interim measure, we utilized Chrome Web to validate the functionality of our code. However, when attempting to run the application on the emulator, a specific error was consistently displayed.

```
FAILURE: Build failed with an exception.

* What went wrong:
Execution failed for task ':flutter_plugin_android_lifecycle:compileDebugJavaWithJavac'.
> Could not resolve all files for configuration ':flutter_plugin_android_lifecycle:androidJdkImage'.
   > Failed to transform core-for-system-modules.jar to match attributes {artifactType=internal_android_jdk_image, org.gradle.libraryelements=jar}
      > Execution failed for JdkImageTransform: C:\Users\88017\AppData\Local\Android\sdk\platforms\android-34\core-for-system-modules.jar.
         > Error while executing process C:\Program Files\Android\Android Studio\jbr\bin\jlink.exe with arguments [--module-path C:\Users\88017\AppData\Local\Android\sdk\platforms\android-34\core-for-system-modules.jar]

* Try:
> Run with --stacktrace option to get the stack trace.
> Run with --info or --debug option to get more log output.
> Run with --scan to get full insights.
> Get more help at https://help.gradle.org.

BUILD FAILED in 48s
Error: Gradle task assembleDebug failed with exit code 1
```

Impact:

Resolving the emulator issue became essential, particularly when integrating images into our app. The process of handling images and videos differs between Android and web platforms, making it necessary to ensure proper functionality on an actual Android environment.

Solution:

After exploring multiple approaches, we discovered a solution in a Git discussion addressing this issue. The resolution involved the following steps:

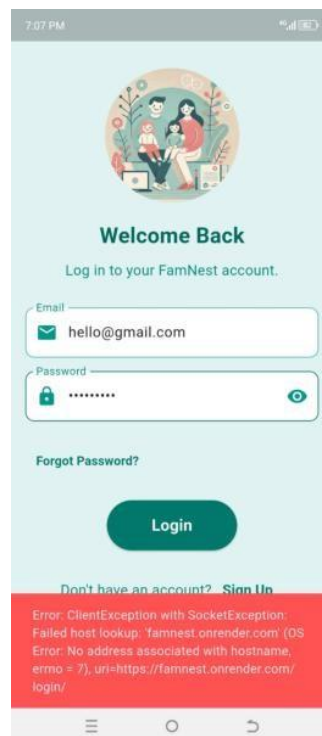
- Updating the **com.android.application** plugin version to **8.3.2** in [android/gradle/setting.gradle](#).
- Upgrading the **Gradle version** to **8.4** and installing the necessary dependencies.

Implementing these changes restored the emulator's functionality, enabling us to proceed with development without further interruptions.

3.1.2 Troubleshooting App Issues on Android Phones:

Issue:

After downloading the Flutter APK to test the app on our Android phone and check if everything was functioning properly, we encountered an error when attempting to log in or sign up.



Impact:

We checked the Wi-Fi connection to ensure there was internet access, but it wasn't the issue. We then rechecked the source code of the login and signup pages and also examined the render, but couldn't find any problems. As a result, we were unable to run our app on the phone due to the issue mentioned earlier.

Solution:

After looking into it for a while, We found a solution: adding user permissions to access the internet and Wi-Fi in the AndroidManifest.xml file. Once we added these permissions, the app ran smoothly on our phones.

This was the solution :



3.1.3 Handling "Null is Not a Subtype of Type 'String'" Error:

Issue:

While working with data retrieved from external sources like MongoDB, we encountered the error "**Null is not a subtype of type 'String'**". This issue occurs because MongoDB is schema-less, meaning fields can be missing or contain null values. Since Flutter enforces strict type safety, attempting to use such data without proper handling results in runtime errors.

Impact:

This error led to unexpected application crashes, negatively impacting the user experience. Additionally, it caused data inconsistencies, where missing or null values resulted in incomplete or incorrect

information being displayed. Debugging and maintaining the affected code became more time-consuming, reducing the app's reliability and making it harder to manage.

Solution:

1. Serializing MongoDB Data in FastAPI

To ensure data integrity, we used **FastAPI** to process and serialize data retrieved from MongoDB before sending it to Flutter. By defining models with **Pydantic**, we enforced a structured data format, performed type validation, and provided default values where necessary.

2. Validating Data Before Use

We implemented validation mechanisms to ensure all API responses were safe to use. Using Dart's `??` operator, we assigned default values to nullable fields, preventing runtime errors.

Example:

```
String name = data?['name'] ?? 'Default Name';
```

3 .Utilizing Dart's Null Safety Features

Dart provides built-in null safety features (`??`, `?.`, and `!`) to handle nullable values safely. We applied:

- `??` to provide fallback values
- `?.` to access properties of nullable objects
- `!` to assert non-nullability when necessary

Example:

```
String name = data?['name'] ?? 'No Name Provided';
```

By implementing these strategies, we effectively mitigated null-related crashes, improved data consistency, and enhanced the overall stability of our application.

3.1.4 Package Compatibility Conflicts:

Issue:

During the development of our Flutter project, we encountered dependency conflicts between packages. These conflicts arise when one package requires a specific version of a dependency that is incompatible with the version needed by another package. As a result, the application failed to compile or function

properly. This issue was particularly noticeable when using packages recommended by language models, such as ChatGPT.

Impact:

The compatibility conflicts impeded the compilation of our application, leading to delays in both development and deployment. Furthermore, these conflicts presented considerable challenges in integrating essential features, as certain packages had to be excluded or replaced. The growing complexity of managing dependencies also affected the long-term maintainability of the project.

Solution:

1. Resolving Version Conflicts Using `dependency_overrides`

To address the version conflicts, we employed the `dependency_overrides` section in the `pubspec.yaml` file. This allowed us to enforce a specific version of a dependency that ensured compatibility across all required packages.

2. Ensuring Package Compatibility Prior to Integration

We proactively checked the compatibility of packages by reviewing official documentation and release notes before incorporating them into the project. Furthermore, we used the `flutter pub deps` command to examine the dependency tree and identify any potential conflicts.

3. Utilizing Alternative or Updated Packages

In cases where compatibility issues persisted, we explored updated versions or alternative packages that offered similar functionality but were more compatible with our project's existing dependencies. This included:

- Upgrading to newer versions of packages that resolved compatibility issues.
- Replacing outdated or unsupported packages with more actively maintained alternatives.

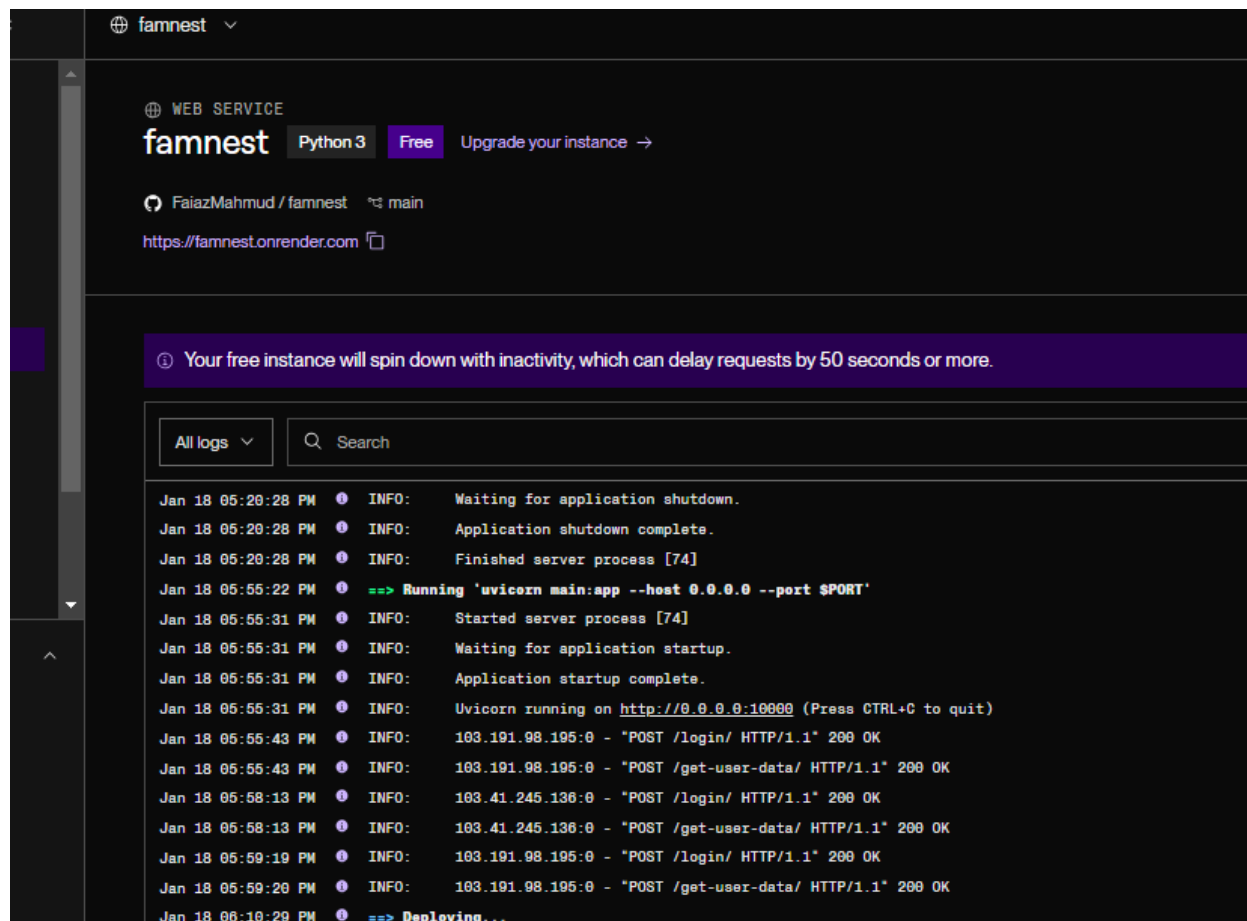
By implementing these approaches, we were able to resolve package compatibility conflicts, thereby enhancing the stability, functionality, and long-term maintainability of the project.

3.1.5 Issues Deploying FastAPI on Render Free Tier :

Issue:

During the deployment of our FastAPI application to Render's free tier, we encountered several challenges that exposed both the advantages and limitations of the platform. These issues included

resource constraints, idle timeouts, and cold starts, all of which impacted the performance and reliability of our application.



Impact:

The application suffered from slower response times due to the limited resources available on the free tier. Additionally, cold starts resulted in delays of 30–50 seconds after periods of inactivity. We also faced occasional deployment errors, and the platform's lack of support for custom domains rendered it unsuitable for production use. This limitation hindered the scalability and branding potential of our application.

Solution:

To mitigate these challenges, we optimized our application to function efficiently within the constraints of Render's free tier by reducing dependencies and implementing effective caching strategies. While these optimizations improved performance, the issues related to idle timeouts and cold starts remained unresolved. As a next step, we plan to explore alternative hosting platforms or consider upgrading to a paid Render plan to achieve better reliability and scalability.

3.1.6 Challenges in Implementing Data Visualization for Budget and Expense Tracking

Issue:

While implementing the month_wise and year_wise analysis page ,we faced a lot of problems implementing the bar and line graphs that were both dynamically updated during runtime or based on data changes.

Impact:

At first, we struggled to display data correctly on the graphs. The initial version showed random placeholder data that didn't update when a user added a budget or expense. This made it difficult to track expenses properly. To fix this, I improved the logic to ensure the graphs updated in real-time with accurate data.

Solution:

Use Data Model : we used the Data Model to manage state efficiently. Within the month_wise_analysis and year_wise_analysis file, we created methods specifically designed to track graph values dynamically, ensuring the graph updated in real-time as data changed.

3.1.7 Optimizing API Response and Error Handling for Document Management:

Issue:

During the implementation of a feature to fetch and display categories and folders for document storage, we observed that the provided FutureBuilder widget code lacked comprehensive error handling and state management. The code did not adequately address scenarios such as network failures, null data, or empty API responses, rendering it unsuitable for robust, real-world application.

Impact:

When the API returned an empty response for categories or folders, the application encountered an unhandled NullPointerException, resulting in a crash. This not only degraded the user experience but also contributed to delays in project progress.

Solution:

To address these shortcomings, we enhanced the FutureBuilder logic by incorporating robust error handling mechanisms and a loading indicator to provide clear feedback during API calls. Additionally, a fallback user interface was implemented to gracefully manage empty responses. The integration of the Provider package for state management further ensured that the document management feature operates reliably and delivers a user-friendly experience.

3.1.8 Inability to Display Images Fetched from URLs

Issue:

During development on our local machine, we initially used local images to display content in our application. However, when we attempted to fetch images from an online URL, the images failed to appear despite a stable internet connection. The image placeholders remained blank even though the image URLs were correctly saved in the database.

Impact:

The updated profile pictures were not displayed correctly within the Circular Avatar, adversely affecting the application's visual presentation and user experience.

Solution:

Upon investigation, we determined that the issue arose because we were using the `Image.file()` widget, which is suitable for local image files. To resolve the problem, we switched to using the `Image.network()` widget for loading images from online sources. This change successfully resolved the issue, and the images are now displayed as intended.

3.1.9 Having trouble fetching images from mongodb atlas

Issue:

While implementing the Time Capsule feature, our idea was to display images in a list format on the screen, one after another. We used ChatGPT to help with the code, but the solution it provided didn't fetch the images correctly from our backend. Although the images were being uploaded and erased as expected, they would disappear as soon as a user navigated to another page.

Impact:

We initially tried using `ListView` to display the images (although we later switched to `GridView`), but nothing seemed to work. We double-checked the connection between the Dart files to make sure everything was set up correctly. It took us quite a while, but eventually, we identified the problem.

Solution:

After reviewing the FastAPI code (`main.py`) and adding multiple print statements, we discovered the issue was with receiving the JSON response from the backend. The contents of the return type (`List<map<String, dynamic>>`) didn't match the structure expected by my Flutter code. Once we fixed

this mismatch, the uploaded images were displayed correctly on the app screen, even after navigating to another page or logging out.

3.1.9 Group Code Trimming in API Requests

Issue

The unique group code used to identify groups was composed of **uppercase and lowercase English letters, numbers, and special characters**. However, during API requests, certain **special characters were being trimmed**, causing incomplete group codes to be sent to the backend.

Impact

- The incorrect group code prevented the **retrieval of group data** from the backend.
- Users were unable to access or manage their groups properly.
- It led to inconsistencies in group identification and data fetching.

Solution

To resolve this issue, we **implemented the unquote library/package**, ensuring that the special characters in the group code were properly encoded and preserved during API requests. This solution allowed the backend to **receive and process the full group code correctly**, restoring seamless group data retrieval.

Issue Resolution & Update

We have successfully **resolved the issue** in the latest APK and have updated it in our **GitHub repository**.



Request: Please check the latest APK to verify the fixes and improvements.

3.2 Lessons Learned

Throughout this project, we gained valuable insights and skills, including:

- **Approaching Real-Life Problems** – Understanding how to analyze and break down complex issues to find effective solutions.
- **Structuring Problem Solutions** – Learning to sketch and outline the framework of a solution before implementation.
- **Frontend Application Design** – Enhancing our ability to create intuitive and visually appealing user interfaces.
- **Working with APIs** – Understanding how to integrate and interact with external data sources through APIs.
- **Backend Development** – Gaining knowledge of how the backend functions and supports application operations.
- **Database Design** – Learning how to structure and manage data efficiently for an application.
- **Team Collaboration** – Developing teamwork and communication skills to work effectively as a group.

3.3 Future Plan

Currently, the app is in its basic version, with only essential features. Moving forward, we aim to enhance both its functionality and user experience. Our key improvement areas include:

- **Improved User Interface** – Refining the design to make the app more visually appealing and user-friendly.
- **Notifications** – Adding real-time alerts to notify users when content is added or updated.
- **PDF Scanner** – Integrating a feature that allows users to quickly scan and create PDFs.
- **Enhanced Data Security** – Strengthening security measures to protect user data.
- **File Search Mechanism** – Implementing an efficient search system to help users locate files easily.
- **Chat feature:** implementing chatting option among the group members